

Full Stack Developer — Step 5

SQL, MongoDB & REST APIs

Roadmap objective: Work with relational and document databases and expose data through APIs

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Relational modeling
- SQL CRUD
- Joins
- MongoDB documents/collections
- REST
- Validation
- API testing
- Choosing a database

Core concepts

1. SQL databases emphasize structured relational data and strong relationships; MongoDB stores JSON-like documents.
2. Choose the data model based on access patterns and consistency needs, not because one database is fashionable.
3. REST APIs should expose resources with clear URLs, HTTP methods and status codes.
4. Validate data at the API boundary before writing to a database.

Concept map

```
SQL, MongoDB & REST APIs
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Relational modeling

Study relational modeling through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. SQL CRUD

Study sql crud through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Joins

Study joins through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. MongoDB documents/collections

Study mongodb documents/collections through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. REST

Study rest through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Validation

Study validation through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

SQL + MongoDB mental model

```
SQL:
careers(id, title, category)
roadmap_steps(id, career_id, step_number, title)
```

```
MongoDB:
{
  "_id": "...",
  "title": "Frontend Developer",
  "skills": ["HTML", "CSS", "JavaScript"]
}
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
SQL	Relational	Tables/relationships
MongoDB	Document	Flexible document model
REST	API style	HTTP resource interface

Common mistakes

- Choosing a database by trend
- Duplicating inconsistent data
- No validation
- Poor REST naming
- No pagination for large collections

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Model one entity in SQL.
- Write down what you expected, what happened, and why.

Lab 2

- Model a document-style version in MongoDB.
- Write down what you expected, what happened, and why.

Lab 3

- Compare read/write access patterns.
- Write down what you expected, what happened, and why.

Lab 4

- Expose a GET endpoint.
- Write down what you expected, what happened, and why.

Lab 5

- Expose a POST endpoint with validation.
- Write down what you expected, what happened, and why.

Lab 6

- Test malformed and missing fields.
- Write down what you expected, what happened, and why.

Mini-project

Build a catalog API backed by MySQL for structured entities and MongoDB for document-style data. Compare the two models and expose selected data through REST.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

SQL vs MongoDB?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a REST resource?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why validate API input?

- Answer structure: definition → example → trade-off/limitation → practical use.

When are joins useful?

- Answer structure: definition → example → trade-off/limitation → practical use.

What makes a good resource URL?

- Answer structure: definition → example → trade-off/limitation → practical use.

How would you choose a database?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Relational	Write this in your own words.	Give one project example.
Document	Write this in your own words.	Give one project example.
Collection	Write this in your own words.	Give one project example.
Schema	Write this in your own words.	Give one project example.
Join	Write this in your own words.	Give one project example.
REST	Write this in your own words.	Give one project example.
Endpoint	Write this in your own words.	Give one project example.
Validation	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Query	Write this in your own words.	Give one project example.
Consistency	Write this in your own words.	Give one project example.

Self-test

- Explain Relational modeling without using its textbook definition.
- Show a small example of SQL CRUD.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Relational modeling.
- I can explain and demonstrate SQL CRUD.
- I can explain and demonstrate Joins.
- I can explain and demonstrate MongoDB documents/collections.
- I can explain and demonstrate REST.
- I can explain and demonstrate Validation.
- I can explain and demonstrate API testing.
- I can explain and demonstrate Choosing a database.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://dev.mysql.com/doc/>
- <https://www.mongodb.com/docs/>
- <https://learning.postman.com/>

Recommended videos

- CodeWithHarry — SQL Tutorial for Beginners (Complete Course using MySQL)
<https://www.youtube.com/watch?v=yE6tIle64tU> (NEW)
- MongoDB Tutorial (Existing DB resource 103)
- REST API Tutorial (Existing DB resource 106)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.